# C++ 面试刷题知识库（按类别背题）

> 用法：**不要从第一页背到最后一页**。按下面「三轮法」走，每类只精刷 3–5 题，题量比题解重要。
>
> 本文档是「知识库」：讲原理、给模板、列必刷题和易错点。 每个专题都有对应的**可编译可运行**的简单版代码，放在仓库的 `src/` 目录里（见下表）， 建议「先读本文的做法，再看 `src/` 的代码，最后合上资料自己手打一遍」。

## 本文档与 `src/` 练习代码的对应关系

| 本文档章节 | 对应练习代码 | 说明 |
| --- | --- | --- |
| 一、数组 / 双指针 / 滑动窗口 | `src/01_two_pointers.cpp` 、 `src/02_sliding_window.cpp` | 对撞双指针、最长/最短型窗口 |
| 二、二分查找 | `src/03_binary_search.cpp` | 整数二分、二分答案、实数二分、三分 |
| 三、哈希表 | `src/04_hash_table.cpp` | 两数之和、分组、前缀和 + 哈希 |
| 四、链表 | `src/05_linked_list.cpp` | 哑节点、快慢指针、就地反转 |
| 五、栈 / 队列 / 单调栈 | `src/06_stack_queue.cpp` | 单调栈、单调队列、双栈队列 |
| 六、二叉树 | `src/07_binary_tree.cpp` | 三种遍历、层序、BST、LCA、建树 |
| 七、回溯 / DFS | `src/08_backtracking.cpp` | 子集 / 组合 / 排列 / N 皇后 |
| 八、图 | `src/09_graph.cpp` | BFS / DFS / 拓扑 / 并查集 / Dijkstra |

| 本文档章节 | 对应练习代码 | 说明 |
| --- | --- | --- |
| 九、堆 / 优先队列 | `src/10_heap.cpp` | 第 K 大、前 K 高频、双堆中位数 |
| 十、动态规划 | `src/11_dynamic_programming.cpp` | 线性 / 背包 / 区间 / 双串 DP |
| 十一、贪心 | `src/12_greedy.cpp` | 区间调度、跳跃游戏、加油站 |
| 十二、字符串 | `src/13_string_kmp.cpp` | KMP、公共前缀、大数相加 |
| 十三、位运算 | `src/14_bit_operations.cpp` | lowbit、异或、比特位计数 |
| 十四、排序 | `src/15_sorting.cpp` | 快排、归并、快速选择、自定义排序 |
| 十五、前缀和 / 差分 | `src/16_prefix_diff.cpp` | 一维 / 二维前缀和、差分 |
| 十六、设计题 | `src/17_design.cpp` | LRU、Trie、O(1) 随机集合 |
| （补充）进阶结构 | `src/18_advanced_structures.cpp` | 树状数组、线段树、离散化 |
| （补充）数学基础 | `src/19_math.cpp` | 快速幂、gcd、组合数、质数筛 |
| （补充）读入输出基本功 | `src/00_basics.cpp` | cin / cout、long long、防溢出 |

> 本文档里的代码模板是「速记版」，注释比 `src/` 里少，命名也更短； 想看清完整注释和运行结果，请直接看 `src/` 里对应的文件。

---

## 零、三轮法：怎么用这份文档

| 轮次 | 目标 | 做法 | 时间 |
| --- | --- | --- | --- |
| 第一轮 | 建立"题型 → 解法"的条件反射 | 每类只做 **2 道**最经典的题，做完立刻合上题解，凭记忆把模板手打一遍 | 2–3 周 |
| 第二轮 | 补盲区 + 提速 | 每类加做 **2–3 道**变体，开始限时（中等题 25 分钟） | 3–4 周 |
| 第三轮 | 面试冲刺 | 只做错题本 + 高频 Top 50，口头讲思路（出声讲，别默看） | 1–2 周 |

**核心原则**：每个类别都要能「不看提示，5 分钟内写出模板骨架」。写不出来就说明还没形成肌肉记忆，回去重打。

---

# 一、数组 / 双指针 / 滑动窗口

## 原理

双指针把 $O(n^2)$ 的枚举压缩成 $O(n)$：**一个指针负责扩大范围，一个负责收缩**。滑动窗口的本质是「维护一个满足条件的区间」。

## 模板：滑动窗口（最长型）

```
// 求满足条件的最长区间长度
int longestWindow(vector<int>& a) {
    int n = a.size(), best = 0;
    int left = 0;
    // 按题目维护窗口状态，例如 sum / cnt 数组 / 去重集合
    long long sum = 0;
    for (int right = 0; right < n; ++right) {
        sum += a[right];              // 1. 右指针进入：更新状态
        while (/* 窗口不合法 */ sum > LIMIT) {
            sum -= a[left];           // 2. 左指针离开：回滚状态
            ++left;
        }
        best = max(best, right - left + 1);  // 3. 此时窗口一定合法
    }
    return best;
}
```

## 模板：滑动窗口（最短型）

```
// 求满足条件的最短区间长度
int shortestWindow(vector<int>& a) {
    int n = a.size(), best = INT_MAX;
    int left = 0;
    long long sum = 0;
    for (int right = 0; right < n; ++right) {
        sum += a[right];
        while (/* 窗口合法 */ sum >= TARGET) {
            best = min(best, right - left + 1);  // 先记录，再收缩
            sum -= a[left];
            ++left;
        }
    }
    return best == INT_MAX ? 0 : best;
}
```

**最长型与最短型的区别只在一处**：最长型在 `while` 结束后记录答案（此时刚变合法），最短型在 `while` 内部记录（此时还合法，再缩就不合法了）。

## 模板：对撞双指针

```
// 有序数组两数之和 / 三数之和 / 盛水容器 都用这个骨架
int twoSum(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int s = a[lo] + a[hi];
        if (s == target) return 1;
        if (s < target) ++lo;   // 太小 -> 左指针右移增大
        else --hi;              // 太大 -> 右指针左移减小
    }
    return 0;
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 3 | 无重复字符的最长子串 | 最长型窗口 + 哈希 |
| 76 | 最小覆盖子串 | 最短型窗口 + 计数 |
| 209 | 长度最小的子数组 | 最短型窗口 |
| 11 | 盛最多水的容器 | 对撞双指针 |
| 15 | 三数之和 | 排序 + 对撞 + 去重 |
| 42 | 接雨水 | 双指针 / 单调栈（两种都要会） |

## 易错点

- 窗口收缩用 `while` 不用 `if` —— 一次可能缩很多步。
- 「最长型」记录答案在 `while` **外**，「最短型」在 `while` **内**。
- 三数之和去重：`if (i > 0 && a[i] == a[i-1]) continue;`，指针移动后也要跳过重复。

---

# 二、二分查找

## 原理

每次砍掉一半搜索空间。**只要能写出单调谓词，就能用同一个函数解决所有变形。**

## 模板：整数二分（一个函数覆盖全部四种）

```
// 在 [lo, hi) 里找第一个使 pred 为 true 的位置。
// pred 必须单调（false...false true...true）。
template <class Predicate>
long long lowerBound(long long lo, long long hi, Predicate pred) {
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;   // 不写 (lo+hi)/2，防溢出
        if (pred(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;   // lo == hi 表示没找到
}
```

四种经典写法的对照（`a` 升序）：

| 需求 | 写法 |
| --- | --- |
| 第一个 `>= x` | `lowerBound(0, n, [&](ll i){ return a[i] >= x; })` |
| 第一个 `> x` | `lowerBound(0, n, [&](ll i){ return a[i] > x; })` |
| 最后一个 `<= x` | `lowerBound(0, n, [&](ll i){ return a[i] > x; }) - 1` |
| 最后一个 `< x` | `lowerBound(0, n, [&](ll i){ return a[i] >= x; }) - 1` |

## 模板：二分答案

```
// 求最小的 t 使 check(t) 成立；答案闭区间 [lo, hi]
long long binaryAnswer(long long lo, long long hi, function<bool(long long)> check) {
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (check(mid)) hi = mid;      // mid 可行 -> 答案在 [lo, mid]
        else lo = mid + 1;             // mid 不可行 -> 答案在 [mid+1, hi]
    }
    return lo;
}
```

**口诀：** `hi = mid` 对应"存在更小的可行解"，`lo = mid + 1` 对应"mid 不满足"。判反了就是死循环或错解。

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 704 | 二分查找 | 最基础 |
| 34 | 在排序数组中查找元素的第一个和最后一个位置 | 找左右边界 |
| 33 | 搜索旋转排序数组 | 断点判断 |
| 153 | 寻找旋转排序数组中的最小值 | `a[mid] > a[hi]` |
| 875 | 爱吃香蕉的珂珂 | 二分答案（最小速度） |
| 410 | 分割数组的最大值 | 二分答案（最小最大值） |

## 易错点

- `mid = lo + (hi - lo) / 2`，永远别写 `(lo + hi) / 2`。
- 区间开闭**全程统一**，不要一会儿 `[lo,hi]` 一会儿 `[lo,hi)`。
- 返回 `lo == hi` 时是"没找到"的哨兵，调用方必须判。
- 二分答案的 `check` 必须真的单调，先在小数据上暴力对拍验证。

---

## 三、哈希表

### 原理

用空间换时间，把"查找"从 O(n) 降到 O(1)。面试里 90% 的用法是**一边遍历一边把"需要的另一半"存起来**。

### 模板：两数之和（边看边存）

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen;   // 值 -> 下标
    for (int i = 0; i < (int)nums.size(); ++i) {
        auto it = seen.find(target - nums[i]);
        if (it != seen.end()) return {it->second, i};
        seen[nums[i]] = i;          // 先查后存，避免用到自己
    }
    return {};
}
```

### 模板：计数 / 分组

```
// 字母异位词分组
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (auto& s : strs) {
        string key = s;
        sort(key.begin(), key.end());   // 排序后的串当 key
        groups[key].push_back(s);
    }
    vector<vector<string>> res;
    for (auto& [k, v] : groups) res.push_back(v);
    return res;
}
```

### 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 1 | 两数之和 | 边看边存 |
| 49 | 字母异位词分组 | 构造 key |
| 128 | 最长连续序列 | 用 set 判断起点 |
| 560 | 和为 K 的子数组 | **前缀和 + 哈希** |
| 146 | LRU 缓存 | 哈希 + 双向链表 |

### 易错点

- `unordered_map` 的 key 必须是可哈希的（`pair` 不行，要自己写 hash）。
- 遍历时插入会**迭代器失效**，先收集再插。
- 「两数之和」必须先查后存，否则 `target = 2*x` 时会用到自己。

---

## 四、链表

### 原理

链表题的通用套路只有三个：**dummy 头节点**、**快慢指针**、**就地反转**。几乎所有题都是这三者的组合。

### 模板：dummy 头（删除 / 合并 / 插入必备）

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};

// 删除倒数第 n 个节点
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode *fast = &dummy, *slow = &dummy;
    for (int i = 0; i <= n; ++i) fast = fast->next;   // fast 先走 n+1 步
    while (fast) { fast = fast->next; slow = slow->next; }
    slow->next = slow->next->next;                     // 此时 slow 停在待删节点的前一个
    return dummy.next;
}
```

## 模板：就地反转

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nxt = head->next;   // 先存后继
        head->next = prev;            // 反转
        prev = head;                  // 前移
        head = nxt;
    }
    return prev;
}
```

## 模板：快慢指针找环

```
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {                // 相遇点
            ListNode* p = head;
            while (p != slow) {            // 从头和相遇点同步走，再遇即入环点
                p = p->next;
                slow = slow->next;
            }
            return p;
        }
    }
    return nullptr;
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 206 | 反转链表 | 就地反转 |
| 92 | 反转链表 II | 局部反转 + dummy |
| 141 / 142 | 环形链表 | 快慢指针 / 入环点 |
| 19 | 删除链表的倒数第 N 个节点 | dummy + 双指针 |
| 21 / 23 | 合并有序链表（K 个） | dummy / 优先队列 |
| 143 | 重排链表 | 找中点 + 反转 + 合并 |

## 易错点

- 凡是要动 `head` 的题，一律先上 `dummy`。
- 反转时先存 `nxt` 再改 `next`，顺序反了就断链。

- 找中点用 `while (fast && fast->next)`，这个条件决定了偶数长度时停在中点还是中点前一个。

---

## 五、栈 / 队列 / 单调栈

### 原理

单调栈维护一个「单调递增或递减」的栈，**每个元素进出各一次**，所以 O(n) 就能求出「每个元素左边/右边第一个比它大/小的元素」。

### 模板：单调栈（下一个更大元素）

```
// res[i] = 右侧第一个比 a[i] 大的元素下标，没有则 -1
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1), st;   // st 存下标，栈内元素值单调递减
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            res[st.back()] = i;   // 此时 a[i] 就是第一个更大的
            st.pop_back();
        }
        st.push_back(i);
    }
    return res;
}
```

**口诀：想找"下一个更大"就用递减栈；想找"下一个更小"就把比较反一下。**栈里存下标（要算距离）还是存值（不要距离）看题目。

### 模板：双栈实现队列 / 用栈做括号匹配

```
// 有效括号
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') || (c == '}' && t !=
        }
    }
    return st.empty();
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 20 | 有效的括号 | 基础栈 |
| 155 | 最小栈 | 辅助栈 |
| 739 | 每日温度 | 单调栈模板题 |
| 84 | 柱状图中最大的矩形 | 单调栈 + 哨兵 |
| 42 | 接雨水 | 单调栈 / 双指针 |
| 232 / 225 | 栈实现队列 / 队列实现栈 | 双栈 |

## 易错点

- 单调栈里忘了 `while` 用 `if`，就只能弹一个。
- 处理"左边也要看"的题（如 84），在数组两端各加一个哨兵 `0` 省掉边界判断。
- 括号匹配要判 `st.empty()`，不能直接 `st.top()`。

---

# 六、二叉树

## 原理

所有二叉树题 = 遍历框架 + 在某个时机做事。**先想清楚"要在前序、中序还是后序做事"**，代码自然就出来了。

## 模板：三种遍历（递归，5 行）

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

void traverse(TreeNode* root) {
    if (!root) return;
    // 前序：在这里处理 root
    traverse(root->left);
    // 中序：在这里处理 root（BST 用这个位置 -> 天然有序）
    traverse(root->right);
    // 后序：在这里处理 root（求高度/路径/子树信息用这个）
}
```

## 模板：迭代中序（面试常考"不用递归"）

```
vector<int> inorder(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }   // 一路向左
        cur = st.top(); st.pop();
        res.push_back(cur->val);
        cur = cur->right;                                // 转向右子树
    }
    return res;
}
```

## 模板：层序 BFS

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = q.size();   // 关键：先记下这一层的节点数
        vector<int> level;
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
```

### 模板： BST 校验（用中序单调性）

```
bool isValidBST(TreeNode* root) {
    TreeNode* prev = nullptr;
    function<bool(TreeNode*)> dfs = [&](TreeNode* node) -> bool {
        if (!node) return true;
        if (!dfs(node->left)) return false;
        if (prev && prev->val >= node->val) return false;   // BST 中序必须严格递增
        prev = node;
        return dfs(node->right);
    };
    return dfs(root);
}
```

### 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 94 / 144 / 145 | 三种遍历（含迭代版） | 框架 |
| 102 | 二叉树的层序遍历 | BFS |
| 104 / 111 | 最大/最小深度 | 后序 |
| 226 | 翻转二叉树 | 递归交换 |
| 101 | 对称二叉树 | 双指针递归 |
| 98 | 验证二叉搜索树 | 中序单调 |
| 236 | 二叉树的最近公共祖先 | 后序回溯 |
| 105 | 从前序与中序构造二叉树 | 分治 + 下标映射 |

### 易错点

- 层序 BFS 必须**先记录 `q.size()`**，边遍历边取 size 会串层。
- 求"最大深度"用后序（自底向上），求"路径"常用前序 + 回溯。
- BST 校验不能只比较父子节点，要传上下界或用中序。

---

# 七、回溯 / DFS

## 原理

回溯 = **决策树的深度优先遍历**。三段式：**做选择 → 递归 → 撤销选择**。写不出回溯，通常是因为没画清决策树。

## 模板：子集 / 组合 / 排列 的统一骨架

```
// 通用回溯骨架
void backtrack(vector<int>& path, vector<bool>& used, vector<vector<int>>& res) {
    if (/* 到达叶子 */) { res.push_back(path); return; }
    for (/* 每个候选 */) {
        if (/* 剪枝：不合法 */) continue;
        path.push_back(x);        // 做选择
        used[x] = true;
        backtrack(path, used, res);
        used[x] = false;
        path.pop_back();          // 撤销选择
    }
}
```

三个经典变体的差异：

| 题型 | 关键差异 |
| --- | --- |
| 子集 | 用 `start` 控制只能往后选，避免重复；`res` 在每个节点都收集 |
| 组合 | 同上，但只在 `path.size() == k` 时收集；可加"剩余不够"剪枝 |
| 排列 | 用 `used[]` 标记，每次从头遍历；`path.size() == n` 时收集 |

## 模板：排列（含去重）

```
vector<vector<int>> permuteUnique(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    int n = nums.size();
    vector<vector<int>> res;
    vector<int> path;
    vector<bool> used(n, false);
    function<void()> dfs = [&]() {
        if ((int)path.size() == n) { res.push_back(path); return; }
        for (int i = 0; i < n; ++i) {
            if (used[i]) continue;
            // 去重核心：同层相同值只用第一个
            if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) continue;
            used[i] = true; path.push_back(nums[i]);
            dfs();
            path.pop_back(); used[i] = false;
        }
    };
    dfs();
    return res;
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 78 | 子集 | 骨架 |
| 46 / 47 | 全排列（含重复） | used + 去重 |
| 39 / 40 | 组合总和（含重复） | 可重复选 / 不可重复 |
| 77 | 组合 | start 控制 |
| 22 | 括号生成 | 剪枝 |
| 51 | N 皇后 | 经典剪枝 |
| 79 | 单词搜索 | 网格 DFS + 回溯 |

### 易错点

- **忘记撤销选择**——这是最常见的 bug，`path.pop_back()` 和 `used[i] = false` 必须成对。
- 去重剪枝 `!used[i-1]` 和 `used[i-1]` 只差一个 `!`，但含义是"同层去重"还是"同枝去重"，会直接决定对错。
- 网格 DFS 别忘了越界和访问标记（可原地改成 `'#'` 省一个数组）。

---

## 八、图（BFS / DFS / 拓扑 / 并查集）

### 原理

图题先分清三件事：**有向还是无向**、**有无权**、**要不要判环**。无权最短路用 BFS，带权非负最短路用 Dijkstra，连通性用并查集。

### 模板：BFS 最短路（无权图）

```
// 从 start 到每个点的最短边数，不可达为 -1
vector<int> bfsShortest(vector<vector<int>>& g, int start) {
    int n = g.size();
    vector<int> dist(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] != -1) continue;   // 已访问
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

## 模板：拓扑排序（判环 + 依赖顺序）

```
vector<int> topoSort(int n, vector<vector<int>>& g) {   // g[u] = u 指向的点
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : g[u]) ++indeg[v];
    queue<int> q;
    for (int i = 0; i < n; ++i) if (indeg[i] == 0) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u])
            if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};   // 有环
    return order;
}
```

## 模板：并查集

```
struct DSU {
    vector<int> p, r;
    explicit DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }   // 路径压缩
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) ++r[a];      // 按秩合并
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
};
```

## 模板：Dijkstra（邻接表 + 优先队列）

```
// g[u] 里存的是 (邻接点, 边权)
vector<long long> dijkstra(int n, const vector<vector<pair<int,int>>>& g, int src) {
    const long long INF = 1000000000000000000LL;   // 10^18，当作无穷大
    vector<long long> dist(n, INF);

    // 小顶堆：堆顶是当前距离最小的点
    using State = pair<long long, int>;            // (距离, 点)
    priority_queue<State, vector<State>, greater<State>> pq;

    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        long long d = pq.top().first;              // 当前距离
        int u = pq.top().second;                   // 当前点
        pq.pop();
        if (d > dist[u]) continue;                 // 过期条目，跳过
        for (const pair<int,int>& edge : g[u]) {
            int v = edge.first;                    // 邻接点
            int w = edge.second;                   // 边权
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 200 | 岛屿数量 | 网格 DFS/BFS |
| 207 / 210 | 课程表 | 拓扑排序判环 |
| 547 | 省份数量 | 并查集 / DFS |

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 743 | 网络延迟时间 | Dijkstra |
| 133 | 克隆图 | DFS + 哈希 |
| 130 | 被围绕的区域 | 反向 DFS（从边界灌） |
| 684 | 冗余连接 | 并查集 |

### 易错点

- BFS 在**入队时**标记已访问，不要在出队时标记，否则会重复入队（超时或死循环）。
- Dijkstra 不能处理负权边；有负权用 Bellman-Ford / SPFA。
- 网格 DFS 递归深度可能很大，`n*m` 上万时改用 BFS 或显式栈。

---

## 九、堆 / 优先队列

### 原理

堆的本质是**只关心当前最大/最小的那个元素**。题目里出现「第 K 大/小」「前 K 个」「动态取最值」，第一反应就应该是堆。

### 模板：求第 K 大 / 前 K 个高频

```
// 前 K 个高频元素：小顶堆维护"当前 K 个最大"
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> cnt;
    for (int x : nums) ++cnt[x];
    // 小顶堆：堆顶是当前 K 个里频率最小的，来了更大的就替换
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    for (auto& [val, c] : cnt) {
        pq.push({c, val});
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
    while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
    return res;
}
```

**口诀：求前 K 大就用小顶堆**（堆里始终保留最大的 K 个，堆顶是最小的那个）。求前 K 小反之用大顶堆。

## 模板：数据流中位数（双堆）

```
class MedianFinder {
    priority_queue<int> small;                                  // 大顶堆，存较小的一半
    priority_queue<int, vector<int>, greater<int>> large;   // 小顶堆，存较大的一半
public:
    void addNum(int num) {
        small.push(num);
        large.push(small.top()); small.pop();
        if (large.size() > small.size()) {                  // 保持 small.size() >= lar
            small.push(large.top()); large.pop();
        }
    }
    double findMedian() {
        if (small.size() > large.size()) return small.top();
        return (small.top() + large.top()) / 2.0;
    }
};
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 215 | 数组中的第 K 个最大元素 | 小顶堆 / 快选 |
| 347 | 前 K 个高频元素 | 堆 + 哈希 |
| 295 | 数据流的中位数 | 双堆 |
| 23 | 合并 K 个升序链表 | 堆 |
| 239 | 滑动窗口最大值 | 单调队列（不是堆，注意区分） |
| 253 | 会议室 II | 小顶堆 |

## 易错点

- `priority_queue<int>` 默认是**大顶堆**；要小顶堆写成 `priority_queue<int, vector<int>, greater<int>>`。
- 自定义类型要写比较器，且容易把大小关系写反（`greater` 对应小顶堆）。

- 滑动窗口最大值用**单调队列** O(n)，用堆是 O(n log n) 会超时。

---

## 十、动态规划

### 原理

DP 三要素：**状态定义、转移方程、边界条件**。面试里 80% 的 DP 卡在「状态定义说不清楚」，不是写不出代码。

### 三步法

1. **状态**：`dp[i]` 表示什么？（必须能用一句话说完整，不能说"表示前 i 个的最优值"这种含糊话）
2. **转移**：`dp[i]` 从哪些更小的状态来？
3. **边界 + 遍历顺序**：`dp[0]` 是多少？从小到大还是从大到小？

### 模板：线性 DP（最大子数组和）

```
int maxSubArray(vector<int>& nums) {
    int best = nums[0], cur = nums[0];
    for (int i = 1; i < (int)nums.size(); ++i) {
        cur = max(nums[i], cur + nums[i]);   // 要么接上前面，要么从自己重新开始
        best = max(best, cur);
    }
    return best;
}
```

### 模板：背包（01 与完全，只差遍历顺序）

```cpp
// 01 背包：每个物品最多选一次
int knapsack01(int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int j = W; j >= w[i]; --j)          // 倒序：保证每件只用一次
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    return dp[W];
}

// 完全背包：每个物品可以选无限次
int knapsackFull(int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int j = w[i]; j <= W; ++j)          // 正序：允许重复选
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    return dp[W];
}
```

**只有遍历顺序一个字符的差别（`j--` vs `j++`），却是两类问题的分水岭。**

## 模板：区间 DP（最长回文子串）

```
string longestPalindrome(string s) {
    int n = s.size();
    vector<vector<bool>> dp(n, vector<bool>(n, false));
    int start = 0, len = 1;
    for (int i = 0; i < n; ++i) dp[i][i] = true;      // 长度 1
    for (int L = 2; L <= n; ++L) {                     // 按长度递增枚举
        for (int i = 0; i + L - 1 < n; ++i) {
            int j = i + L - 1;
            if (s[i] != s[j]) continue;
            dp[i][j] = (L == 2) ? true : dp[i + 1][j - 1];
            if (dp[i][j] && L > len) { start = i; len = L; }
        }
    }
    return s.substr(start, len);
}
```

## 模板：二维路径 DP（不同路径 / 最小路径和）

```
int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    vector<vector<int>> dp(m, vector<int>(n, 0));
    dp[0][0] = grid[0][0];
    for (int i = 1; i < m; ++i) dp[i][0] = dp[i-1][0] + grid[i][0];   // 第一列
    for (int j = 1; j < n; ++j) dp[0][j] = dp[0][j-1] + grid[0][j];   // 第一行
    for (int i = 1; i < m; ++i)
        for (int j = 1; j < n; ++j)
            dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j];
    return dp[m-1][n-1];
}
```

## 必刷题（按难度递进）

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 70 | 爬楼梯 | 入门，滚动数组 |

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 53 | 最大子数组和 | 线性 DP |
| 198 / 213 | 打家劫舍（含环形） | 状态定义 |
| 300 | 最长递增子序列 | $O(n^2)$ + 贪心二分 O(n log n) |
| 322 | 零钱兑换 | 完全背包变体 |
| 416 | 分割等和子集 | 01 背包判定 |
| 1143 | 最长公共子序列 | 双串 DP |
| 72 | 编辑距离 | 双串 DP + 三选一 |
| 5 | 最长回文子串 | 区间 DP |
| 5 → 32 | 最长有效括号 | 线性 DP 难点 |

### 易错点

- 01 背包内层**必须倒序**，正序就变成完全背包了。
- 区间 DP 按「区间长度」递增枚举，不是按左端点。
- 双串 DP 注意下标错位（`s[i-1]` vs `s[i]`），`dp` 多开一行一列当空串。
- 空间可以优化时（滚动数组），注意优化后遍历方向是否需要跟着变。

---

## 十一、贪心

### 原理

每一步都选当前最优，并且**能证明局部最优推得出全局最优**。面试里判断能否用贪心的关键是：**能不能举出一个反例**。

### 模板：区间调度（按结束时间排序）

```
// 最多能参加多少个不重叠的会议
int maxMeetings(vector<vector<int>>& meetings) {
    sort(meetings.begin(), meetings.end(),
         [](const vector<int>& a, const vector<int>& b) { return a[1] < b[1]; });  // 
    int cnt = 0, lastEnd = INT_MIN;
    for (auto& m : meetings) {
        if (m[0] >= lastEnd) { ++cnt; lastEnd = m[1]; }
    }
    return cnt;
}
```

## 模板：跳跃游戏（维护最远可达）

```
bool canJump(vector<int>& nums) {
    int reach = 0;
    for (int i = 0; i < (int)nums.size(); ++i) {
        if (i > reach) return false;        // 走不到这里
        reach = max(reach, i + nums[i]);
    }
    return true;
}
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 55 / 45 | 跳跃游戏 I / II | 维护最远可达 |
| 435 | 无重叠区间 | 按结束排序 |
| 452 | 用最少数量的箭引爆气球 | 区间贪心 |
| 121 | 买卖股票的最佳时机 | 维护最小前缀 |
| 134 | 加油站 | 遍历 + 起点重置 |
| 763 | 划分字母区间 | 记录最远位置 |

### 易错点

- 区间问题排序依据：求「最多不重叠」按**结束**排，求「合并区间」按**开始**排。排错了就错。
- 贪心只能给结论，面试时要能说出「为什么这样选不会更差」。

---

## 十二、字符串

### 原理

C++ 字符串题的两个高频坑：**字符类型**（`'a'` 是 char，加减要转 int）和**子串 vs 子序列**（连续 vs 不连续）。

### 模板：KMP（面试偶尔考，会写就加分）

```
// 求 pattern 在 text 中所有出现位置
vector<int> kmpSearch(const string& text, const string& pattern) {
    int n = text.size(), m = pattern.size();
    if (m == 0) return {};
    vector<int> nxt(m, 0);
    for (int i = 1, j = 0; i < m; ++i) {          // 构建 next
        while (j > 0 && pattern[i] != pattern[j]) j = nxt[j - 1];
        if (pattern[i] == pattern[j]) ++j;
        nxt[i] = j;
    }
    vector<int> res;
    for (int i = 0, j = 0; i < n; ++i) {          // 匹配
        while (j > 0 && text[i] != pattern[j]) j = nxt[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == m) { res.push_back(i - m + 1); j = nxt[j - 1]; }
    }
    return res;
}
```

### 模板：字符串常用操作速查

```
string s = "hello";
s.substr(1, 3);                  // "ell"（起点, 长度）
s.find("ll");                    // 2，找不到返回 string::npos
reverse(s.begin(), s.end());     // 原地反转
to_string(123);                  // int -> "123"
stoi("123");                     // "123" -> int（stoll 转 long long）
transform(s.begin(), s.end(), s.begin(), ::tolower);   // 转小写
// 判断字符
isdigit(c); isalpha(c); islower(c); isupper(c); isalnum(c);
// 拼字符串用 += 或 ostringstream，别在循环里 s = s + x（O(n²)）
```

### 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 5 | 最长回文子串 | 中心扩展 / 区间 DP |
| 151 | 反转字符串中的单词 | 双指针 |
| 28 | 找出字符串中第一个匹配项 | KMP / find |
| 3 | 无重复字符的最长子串 | 滑动窗口 |
| 415 | 字符串相加 | 模拟进位 |
| 43 | 字符串相乘 | 大数乘法 |

### 易错点

- `s.size()` 返回 `size_t`（无符号），`s.size() - 1` 在 `s` 为空时会变成巨大正数。统一用 `(int)s.size()`。
- `substr(pos, len)` 的第二个参数是**长度**不是结束下标。
- 大数运算别转 `long long`，直接模拟。

---

## 十三、位运算

## 原理

位运算是 O(1) 的"小技巧"题，掌握 6 个操作就够应付面试。

## 模板：常用位操作

```
n & 1              // 判奇偶
n >> 1             // 除以 2
n & (n - 1)         // 消掉最低位的 1（求二进制里 1 的个数、判 2 的幂）
n & (-n)            // 取出最低位的 1（lowbit）
n ^ n = 0          // 异或：相同为 0，不同为 1（找只出现一次的数）
n ^ 0 = n
```

```
// 统计二进制中 1 的个数
int popcount(int n) {
    int cnt = 0;
    while (n) { n &= n - 1; ++cnt; }   // 每次消掉一个 1
    return cnt;
}

// 只出现一次的数字（其余都出现两次）
int singleNumber(vector<int>& nums) {
    int x = 0;
    for (int v : nums) x ^= v;         // 相同的抵消
    return x;
}

// 判断是否是 2 的幂
bool isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 136 | 只出现一次的数字 | 异或 |

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 191 | 位 1 的个数 | `n & (n-1)` |
| 231 | 2 的幂 | `n & (n-1)` |
| 338 | 比特位计数 | DP + lowbit |
| 260 | 只出现一次的数字 III | 异或 + 分组 |
| 461 | 汉明距离 | 异或 + popcount |

### 易错点

- C++ 里对负数做 `>>` 是**算术右移**（补 1），不是逻辑右移。要逻辑右移用 `unsigned`。
- `n & (n - 1)` 在 `n = 0` 时是 `0 & -1 = 0`，判 2 的幂前必须先排除 `n <= 0`。

---

## 十四、排序

### 原理

面试要求手写的只有两个：**快排**和**归并**。其他场景直接调 `std::sort`。

### 模板：快速排序

```
void quickSort(vector<int>& a, int lo, int hi) {   // [lo, hi]
    if (lo >= hi) return;
    int pivot = a[lo + (hi - lo) / 2];             // 取中间值当基准，避免有序数组退化
    int i = lo, j = hi;
    while (i <= j) {
        while (a[i] < pivot) ++i;
        while (a[j] > pivot) --j;
        if (i <= j) { swap(a[i], a[j]); ++i; --j; }
    }
    quickSort(a, lo, j);
    quickSort(a, i, hi);
}
```

## 模板：归并排序（顺带求逆序对）

```
long long mergeSort(vector<int>& a, vector<int>& tmp, int lo, int hi) {   // [lo, hi]
    if (lo >= hi) return 0;
    int mid = lo + (hi - lo) / 2;
    long long cnt = mergeSort(a, tmp, lo, mid) + mergeSort(a, tmp, mid + 1, hi);
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { tmp[k++] = a[j++]; cnt += mid - i + 1; }   // 右边更小 -> 逆序对
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    for (int t = lo; t <= hi; ++t) a[t] = tmp[t];
    return cnt;
}
```

## 模板：自定义排序（面试高频）

```
// 把数组排成最小的数：按 "ab" < "ba" 排序
string minNumber(vector<int>& nums) {
    vector<string> ss;
    for (int x : nums) ss.push_back(to_string(x));
    sort(ss.begin(), ss.end(), [](const string& a, const string& b) {
        return a + b < b + a;
    });
    string res;
    for (auto& s : ss) res += s;
    return res;
}
```

### 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 912 | 排序数组 | 快排 / 归并手写 |
| 215 | 第 K 个最大元素 | 快速选择 O(n) |
| 56 | 合并区间 | 按起点排序 |
| 179 / 剑指45 | 最大数 / 把数组排成最小的数 | 自定义比较器 |
| 493 | 翻转对 | 归并 + 计数 |
| 148 | 排序链表 | 归并（链表版） |

### 易错点

- 快排基准取**首元素**在有序数据上会退化 $O(n^2)$，必须取中间或随机。
- `sort` 的比较器必须满足**严格弱序**（`a < b` 且 `b < a` 不能同时为真），写 `<=` 会导致未定义行为甚至段错误。
- 归并排序的 `tmp` 数组要在最外层分配好传进去，别在递归里每次新建。

---

## 十五、前缀和 / 差分

## 原理

**前缀和**把区间求和变成 O(1)；**差分**把区间加变成 O(1)，最后一遍前缀和还原。

## 模板：一维 + 二维前缀和

```
// 一维：pre[i] = a[0..i-1] 的和，区间 [l, r] 和 = pre[r+1] - pre[l]
vector<long long> buildPrefix(vector<int>& a) {
    int n = a.size();
    vector<long long> pre(n + 1, 0);
    for (int i = 0; i < n; ++i) pre[i + 1] = pre[i] + a[i];
    return pre;
}

// 二维：pre[i][j] = 左上角 (0,0) 到 (i-1,j-1) 的和
vector<vector<long long>> buildPrefix2D(vector<vector<int>>& g) {
    int m = g.size(), n = g[0].size();
    vector<vector<long long>> pre(m + 1, vector<long long>(n + 1, 0));
    for (int i = 1; i <= m; ++i)
        for (int j = 1; j <= n; ++j)
            pre[i][j] = g[i-1][j-1] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];
    return pre;
}
// 查询 (r1,c1) 到 (r2,c2)（0-indexed，含端点）：
// pre[r2+1][c2+1] - pre[r1][c2+1] - pre[r2+1][c1] + pre[r1][c1]
```

## 模板：差分（区间加）

```
// 对 [l, r] 全部加 v，最后还原原数组
void rangeAdd(vector<long long>& diff, int l, int r, long long v) {
    diff[l] += v;
    diff[r + 1] -= v;     // 注意 r+1 可能越界，数组多开一位
}
// 还原：for (int i = 1; i < n; ++i) diff[i] += diff[i-1];
```

### 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 560 | 和为 K 的子数组 | 前缀和 + 哈希 |
| 303 | 区域和检索 | 一维前缀和 |
| 304 | 二维区域和检索 | 二维前缀和 |
| 1109 | 航班预订统计 | 差分模板题 |
| 2132 | 用邮票贴满网格图 | 二维差分 |
| 238 | 除自身以外数组的乘积 | 前后缀积 |

### 易错点

- 前缀和数组开 `n + 1`，`pre[0] = 0`，这样 `[l, r]` 的公式不用特判 `l = 0`。
- 二维前缀和的容斥符号：`+ - - +`，写反了答案就错。
- 差分的 `r + 1` 要判越界，或者数组多开一位。

---

## 十六、设计题

### 原理

设计题（LRU、LFU、时间轮、限流器）考的是**把数据结构组合起来**。套路固定：**哈希表给 O(1) 定位，链表/堆维护顺序**。

### 模板：LRU 缓存（哈希 + 双向链表，必背）

```
class LRUCache {
    struct Node {
        int key, val;
        Node *prev = nullptr, *next = nullptr;
        Node(int k, int v) : key(k), val(v) {}
    };
    int cap;
    unordered_map<int, Node*> mp;   // key -> 节点
    Node *head, *tail;               // 哨兵：head 侧最新，tail 侧最旧

    void remove(Node* node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }
    void addFront(Node* node) {      // 插到 head 之后 = 最新
        node->next = head->next;
        node->prev = head;
        head->next->prev = node;
        head->next = node;
    }
public:
    LRUCache(int capacity) : cap(capacity) {
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head->next = tail;
        tail->prev = head;
    }
    int get(int key) {
        auto it = mp.find(key);
        if (it == mp.end()) return -1;
        Node* node = it->second;
        remove(node); addFront(node);   // 访问即刷新
        return node->val;
    }
    void put(int key, int value) {
        auto it = mp.find(key);
```

```
        if (it != mp.end()) {
            Node* node = it->second;
            node->val = value;
            remove(node); addFront(node);
            return;
        }
        if ((int)mp.size() == cap) {
            Node* oldest = tail->prev;   // 淘汰最旧
            remove(oldest);
            mp.erase(oldest->key);
            delete oldest;
        }
        Node* node = new Node(key, value);
        mp[key] = node;
        addFront(node);
    }
};
```

## 必刷题

| 题号 | 题目 | 考点 |
| --- | --- | --- |
| 146 | LRU 缓存 | 哈希 + 双向链表 |
| 460 | LFU 缓存 | 哈希 + 频率桶 |
| 380 | O(1) 时间插入、删除和获取随机元素 | 哈希 + 数组 |
| 295 | 数据流的中位数 | 双堆 |
| 355 | 设计推特 | 哈希 + 堆 |
| 208 | 实现 Trie | 前缀树 |

## 易错点

- 哨兵节点能消掉大量 `nullptr` 判断，别省。
- `put` 更新已有 key 时别忘了刷新位置（移到最新）。
- 淘汰时 `mp.erase` 和 `delete` 都要做，只删一个就内存泄漏或悬空指针。

## 附录 A：按面试频率排序的 Top 30（冲刺用）

如果时间只剩一周，就刷这 30 道：

| 排名 | 题号 | 题目 | 类别 |
|---|---|---|---|
| 1 | 3 | 无重复字符的最长子串 | 滑动窗口 |
| 2 | 206 | 反转链表 | 链表 |
| 3 | 146 | LRU 缓存 | 设计 |
| 4 | 215 | 第 K 个最大元素 | 堆 |
| 5 | 15 | 三数之和 | 双指针 |
| 6 | 102 | 层序遍历 | 二叉树 |
| 7 | 53 | 最大子数组和 | DP |
| 8 | 21 | 合并两个有序链表 | 链表 |
| 9 | 33 | 搜索旋转排序数组 | 二分 |
| 10 | 200 | 岛屿数量 | 图 DFS |
| 11 | 5 | 最长回文子串 | 字符串 / DP |
| 12 | 70 | 爬楼梯 | DP |
| 13 | 94 | 二叉树中序遍历 | 二叉树 |
| 14 | 1 | 两数之和 | 哈希 |
| 15 | 141 | 环形链表 | 快慢指针 |
| 16 | 236 | 最近公共祖先 | 二叉树 |
| 17 | 322 | 零钱兑换 | 背包 |
| 18 | 121 | 买卖股票的最佳时机 | 贪心 |
| 19 | 46 | 全排列 | 回溯 |
| 20 | 20 | 有效的括号 | 栈 |

| 排名 | 题号 | 题目 | 类别 |
| --- | --- | --- | --- |
| 21 | 42 | 接雨水 | 单调栈 |
| 22 | 239 | 滑动窗口最大值 | 单调队列 |
| 23 | 300 | 最长递增子序列 | DP |
| 24 | 207 | 课程表 | 拓扑 |
| 25 | 56 | 合并区间 | 排序 |
| 26 | 4 | 寻找两个正序数组的中位数 | 二分（困难） |
| 27 | 23 | 合并 K 个升序链表 | 堆 |
| 28 | 105 | 前序中序构造二叉树 | 分治 |
| 29 | 72 | 编辑距离 | DP |
| 30 | 208 | 实现 Trie | 设计 |

---

## 附录 B：C++ 面试常被问的语言细节

| 话题 | 必须能答上来的点 |
| --- | --- |
| `vector` vs `list` | 连续内存 vs 链表；随机访问 O(1) vs O(n)；缓存友好度 |
| `reserve` / `resize` | `reserve` 只改容量不改 size；`resize` 改 size 并构造元素 |
| 迭代器失效 | `vector` 扩容后全部失效；`unordered_map` rehash 后迭代器失效但指针/reference 不失效 |
| `new` / `delete` 与 `malloc` / `free` | 前者调构造/析构，后者只管内存 |
| 智能指针 | `unique_ptr` 独占、`shared_ptr` 引用计数（`make_shared` 一次分配）、`weak_ptr` 破循环引用 |
| `const` | `const int* p`（指向常量）vs `int* const p`（常量指针）vs `const int* const p` |
| 移动语义 | `std::move` 只是类型转换，真正移动发生在移动构造/赋值里 |

| 话题 | 必须能答上来的点 |
| --- | --- |
| 虚函数 | 虚表指针、动态绑定、构造函数不能是虚函数、析构函数应该是虚函数 |
| 内存对齐 | 结构体大小按最大成员对齐，`#pragma pack` 可改 |
| `static` | 局部静态（首次初始化，C++11 起线程安全）、全局静态（内部链接）、类静态成员 |

---

## 附录 C：刷题时的自查清单（每道题做完过一遍）

1. **边界**：空输入、单元素、全相同、负数、`INT_MAX` 附近的溢出？
2. **复杂度**：时间/空间都说得出吗？能不能再优化一档？
3. **能不能口述**：合上代码，用 3 句话讲清「思路 → 关键步骤 → 复杂度」。
4. **有没有更简单的解法**：暴力能不能过？面试官更想看哪种？
5. **代码风格**：变量命名清晰，没有魔法数字，没有为了通过而硬凑的特判。

---

## 附录 D：按周排的计划（12 周）

| 周 | 类别 | 每日题量 |
| --- | --- | --- |
| 1 | 数组 / 双指针 / 滑动窗口 | 2 题 |
| 2 | 二分 + 哈希 | 2 题 |
| 3 | 链表 + 栈 | 2 题 |
| 4 | 二叉树（重点） | 3 题 |
| 5 | 回溯 / DFS | 2 题 |
| 6 | 图（BFS / 拓扑 / 并查集） | 2 题 |
| 7 | 堆 + 贪心 | 2 题 |
| 8 | 动态规划（一）：线性 / 背包 | 2 题 |
| 9 | 动态规划（二）：区间 / 双串 | 2 题 |
| 10 | 字符串 + 位运算 + 排序 | 2 题 |

| 周 | 类别 | 每日题量 |
| --- | --- | --- |
| 11 | 设计题 + Top 30 冲刺 | 3 题 |
| 12 | 全部错题重做 + 模拟面试 | 复习 |

**每天固定 30 分钟做一件事**：合上所有资料，手打当天类别的模板骨架。写不出来就是没记住。